

Spring Boot 3

The first two Spring Boot documents covered the container and the database layer. This one builds a REST API end to end, in the shape a working team would actually write it.

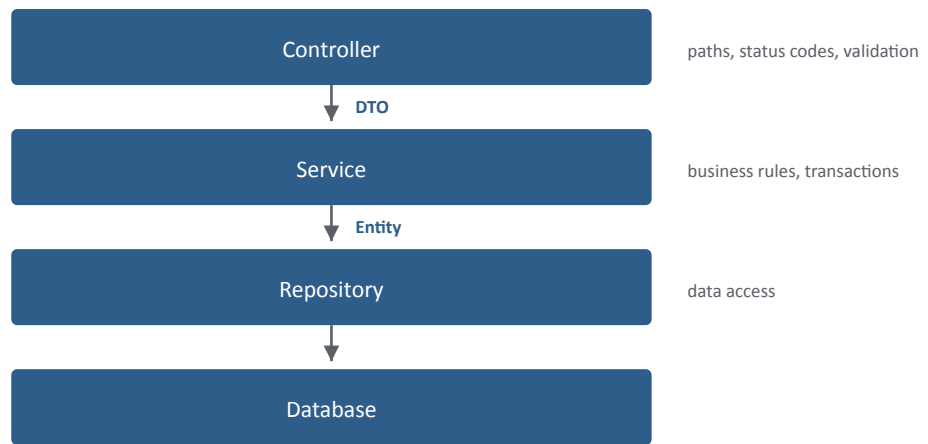
One example runs through the whole document: an employee API with five endpoints.

Contents

1. [The layers](#)
2. [The entity and the DTOs](#)
3. [The repository](#)
4. [The service](#)
5. [The controller](#)
6. [Validation](#)
7. [Error handling](#)
8. [Testing](#)
9. [Making it production ready](#)
10. [Where Spring Security sits](#)
11. [Configuring it](#)
12. [Running as a resource server](#)
13. [Users and passwords](#)
14. [Method security](#)
15. [Testing it](#)

1. The layers

A REST service is three layers, and each one has a single job.



The controller never sees an entity and the repository never sees a DTO. The service is where the two meet.

Layer	Its job	What it handles
Controller	Speaks HTTP	Paths, status codes, request validation
Service	Holds the rules	Business logic, transaction boundaries
Repository	Reaches the database	Entities and queries

Returning the entity straight from the controller is the shortcut to avoid. It ties the public API to the database schema, so renaming a column becomes a breaking change for every client. It also serializes whatever associations happen to be loaded, which varies by code path.

2. The entity and the DTOs

The entity is the database shape:

```

@Entity
@Table(name = "employees")
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 50)
    private String name;

    @Column(nullable = false, unique = true)
    private String email;

    @Column(precision = 10, scale = 2)
    private BigDecimal salary;

    protected Employee() { }

    // constructor, getters and setters
}

```

The DTOs are the API shape, and records suit them exactly:

```

public record EmployeeRequest(
    @NotBlank @Size(max = 50) String name,
    @NotBlank @Email String email,
    @NotNull @Positive BigDecimal salary) { }

public record EmployeeResponse(
    Long id,
    String name,
    String email,
    BigDecimal salary) { }

```

Two DTOs, not one. The request has no `id`, because the client does not choose it. The response carries no validation annotations, because nothing validates a response. One shared class would make every field optional in one direction and required in the other.

Something has to convert between them:

```

@Component
public class EmployeeMapper {

    public Employee toEntity(EmployeeRequest request) {
        return new Employee(request.name(), request.email(), request.salary());
    }

    public EmployeeResponse toResponse(Employee employee) {
        return new EmployeeResponse(
            employee.getId(),
            employee.getName(),
            employee.getEmail(),
            employee.getSalary());
    }
}

```

Written by hand it is obvious and boring, which is the right quality here. Once there are twenty of them, MapStruct generates the same code at compile time.

3. The repository

```

public interface EmployeeRepository extends JpaRepository<Employee, Long> {

    boolean existsByEmail(String email);

    Page<Employee> findByDepartmentId(Long departmentId, Pageable pageable);
}

```

Nothing more belongs here. Anything that decides something belongs in the service.

4. The service

```
@Service
@Transactional(readOnly = true)
public class EmployeeService {

    private final EmployeeRepository repository;
    private final EmployeeMapper mapper;

    public EmployeeService(EmployeeRepository repository, EmployeeMapper mapper) {
        this.repository = repository;
        this.mapper = mapper;
    }

    public Page<EmployeeResponse> findAll(Pageable pageable) {
        return repository.findAll(pageable).map(mapper::toResponse);
    }

    public EmployeeResponse findById(Long id) {
        return repository.findById(id)
            .map(mapper::toResponse)
            .orElseThrow(() -> new EmployeeNotFoundException(id));
    }
}
```

The three methods that write, in the same class:

```

@Transactional
public EmployeeResponse create(EmployeeRequest request) {
    if (repository.existsByEmail(request.email())) {
        throw new DuplicateEmailException(request.email());
    }
    return mapper.toResponse(repository.save(mapper.toEntity(request)));
}

@Transactional
public EmployeeResponse update(Long id, EmployeeRequest request) {
    Employee employee = repository.findById(id)
        .orElseThrow(() -> new EmployeeNotFoundException(id));

    employee.setName(request.name());
    employee.setEmail(request.email());
    employee.setSalary(request.salary());

    return mapper.toResponse(employee);    // dirty checking writes it
}

@Transactional
public void delete(Long id) {
    if (!repository.existsById(id)) {
        throw new EmployeeNotFoundException(id);
    }
    repository.deleteById(id);
}

```

Three habits in that class are worth naming.

`@Transactional(readonly = true)` sits on the class and the three writing methods override it. Reads then skip dirty checking by default, and a method that writes has to say so.

The service throws `EmployeeNotFoundException`, not `ResponseStatusException`. A service that knows about HTTP cannot be called from a scheduled job or a message listener.

`update` changes the managed entity and never calls `save`. Dirty checking writes it at commit.

5. The controller

```
@RestController
@RequestMapping("/api/v1/employees")
public class EmployeeController {

    private final EmployeeService service;

    public EmployeeController(EmployeeService service) {
        this.service = service;
    }

    @GetMapping
    public Page<EmployeeResponse> list(@PageableDefault(size = 20) Pageable pageable)
    {
        return service.findAll(pageable);
    }

    @GetMapping("/{id}")
    public EmployeeResponse get(@PathVariable Long id) {
        return service.findById(id);
    }
}
```

And the three that change something:

```
@PostMapping
public ResponseEntity<EmployeeResponse> create(
    @Valid @RequestBody EmployeeRequest request) {

    EmployeeResponse created = service.create(request);
    return ResponseEntity
        .created(URI.create("/api/v1/employees/" + created.id()))
        .body(created);
}

@PutMapping("/{id}")
public EmployeeResponse update(@PathVariable Long id,
    @Valid @RequestBody EmployeeRequest request) {
    return service.update(id, request);
}

@DeleteMapping("/{id}")
@ResponseStatus(HttpStatus.NO_CONTENT)
public void delete(@PathVariable Long id) {
    service.delete(id);
}
```

There is no logic and no `try / catch` anywhere in it. That is the point.

Creating an employee:

```
POST /api/v1/employees HTTP/1.1
Content-Type: application/json

{ "name": "Ravi", "email": "ravi@example.com", "salary": 75000.00 }
```

```
HTTP/1.1 201 Created
Location: /api/v1/employees/1
Content-Type: application/json

{ "id": 1, "name": "Ravi", "email": "ravi@example.com", "salary": 75000.00 }
```

Four details there are deliberate:

Detail	Why
<code>/api/v1/</code>	The version is in the path from the first release, not added later
<code>201</code> with <code>Location</code>	The client learns where the new resource lives
<code>204</code> on delete	Success with nothing to return
Plural nouns, no verbs	<code>/employees/1</code> , not <code>/getEmployee?id=1</code>

6. Validation

The annotations live on the request record, and `@Valid` on the parameter turns them on. A failure is raised before the controller body runs.

Annotation	Checks
<code>@NotNull</code>	Not null
<code>@NotBlank</code>	Not null, and not only whitespace
<code>@NotEmpty</code>	Not null and not empty, for collections
<code>@Size(max = 50)</code>	Length or size
<code>@Email</code>	The shape of an email address
<code>@Positive</code> , <code>@Min</code> , <code>@Max</code>	Numeric range
<code>@Past</code> , <code>@Future</code>	Dates

It needs its own starter, which the web starter has not included since Spring Boot 2.3:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

@Valid and @Validated

Two annotations, and the difference is asked about often.

	@Valid	@Validated
Comes from	Jakarta Bean Validation	Spring
Supports groups	No	Yes
On a method parameter	Yes	Yes
On a class, to validate its method arguments	No	Yes
Failure raises	MethodArgumentNotValidException	ConstraintViolationException

That last row explains a common confusion. The same broken rule produces a different exception depending on where validation was triggered, so an error handler has to expect both.

Nested objects and collections

Validation does not travel into a nested object on its own. The field needs `@Valid` as well:

```
public record EmployeeRequest(
    @NotBlank String name,
    @NotNull @Valid AddressRequest address,
    @NotEmpty List<@Valid SkillRequest> skills) { }
```

Without `@Valid` on `address`, its own `@NotBlank` fields are never checked. The `@Valid` inside the generic type is what reaches each element of the list.

Validation groups

One record often serves two operations with different rules. A group says which rules apply when:

```

public interface OnCreate { }
public interface OnUpdate { }

public record EmployeeRequest(
    @Null(groups = OnCreate.class)
    @NotNull(groups = OnUpdate.class)
    Long id,

    @NotBlank(groups = {OnCreate.class, OnUpdate.class})
    String name) { }

```

```

@PostMapping
public EmployeeResponse create(
    @Validated(OnCreate.class) @RequestBody EmployeeRequest request) { ... }

```

Groups need `@Validated`, since `@Valid` cannot carry them.

A custom constraint

Anything the built-in annotations cannot express becomes an annotation of its own. Two pieces: the annotation, and the class that checks it.

```

@Target({ElementType.FIELD, ElementType.RECORD_COMPONENT})
@Retention(RetentionPolicy.RUNTIME)
@Constraint(validatedBy = WorkEmailValidator.class)
public @interface WorkEmail {
    String message() default "must be a company address";
    Class<?>[] groups() default {};
    Class<? extends Payload>[] payload() default {};
}

```

```

public class WorkEmailValidator implements ConstraintValidator<WorkEmail, String> {

    @Override
    public boolean isValid(String value, ConstraintValidatorContext context) {
        return value == null || value.endsWith("@example.com");
    }
}

```

```

public record EmployeeRequest(@NotBlank @Email @WorkEmail String email) { }

```

Returning true for null is deliberate. A constraint checks one thing, and whether the value may be absent is `@NotNull`'s job. Mixing them means every field needs its own null handling.

A rule that compares two fields goes on the class instead of a field, because no single field can see the other:

```
@ValidDateRange // a class-level constraint on the record itself
public record LeaveRequest(LocalDate from, LocalDate to) { }
```

Validating service methods

Validation is not only for controllers. `@Validated` on a class turns on checking of its method arguments:

```
@Service
@Validated
public class EmployeeService {

    public EmployeeResponse findByEmail(@NotBlank @Email String email) { ... }
}
```

That guards a service called from a scheduled job or a message listener, where no controller ran. It throws `ConstraintViolationException`, which is why the handler in the next section catches that as well.

7. Error handling

Every expected failure becomes a status code in one place, and no controller catches anything.

```

@RestControllerAdvice
public class ApiExceptionHandler {

    @ExceptionHandler(EmployeeNotFoundException.class)
    public ProblemDetail handleNotFound(EmployeeNotFoundException e) {
        ProblemDetail problem =
            ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND,
e.getMessage());
        problem.setTitle("Employee not found");
        return problem;
    }

    @ExceptionHandler(DuplicateEmailException.class)
    public ProblemDetail handleDuplicate(DuplicateEmailException e) {
        return ProblemDetail.forStatusAndDetail(HttpStatus.CONFLICT, e.getMessage());
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ProblemDetail handleValidation(MethodArgumentNotValidException e) {
        ProblemDetail problem = ProblemDetail.forStatus(HttpStatus.BAD_REQUEST);
        problem.setTitle("Validation failed");
        problem.setProperty("errors", e.getBindingResult().getFieldErrors().stream()
            .collect(toMap(FieldError::getField,
FieldError::getDefaultMessage)));
        return problem;
    }
}

```

`ProblemDetail` arrived in Spring 6 and implements RFC 7807, the standard shape for an API error. A client gets the same structure from every endpoint:

```

HTTP/1.1 400 Bad Request
Content-Type: application/problem+json

{
  "type": "about:blank",
  "title": "Validation failed",
  "status": 400,
  "errors": {
    "email": "must be a well-formed email address",
    "salary": "must be greater than 0"
  }
}

```

A stack trace must never reach a client. Anything unexpected becomes a plain 500 with an identifier, and the detail goes to the log where that identifier can find it.

Spring's own exceptions

Most failures are raised by Spring before any application code runs. Each has a sensible default status, and each is worth a handler when the message matters.

Exception	Raised when	Status
<code>MethodArgumentNotValidException</code>	<code>@Valid</code> fails on a body	400
<code>ConstraintViolationException</code>	<code>@Validated</code> fails on a parameter	400
<code>HttpMessageNotReadableException</code>	The JSON is malformed or a type will not parse	400
<code>MethodArgumentTypeMismatchException</code>	A path or query value will not convert	400
<code>MissingServletRequestParameterException</code>	A required parameter is absent	400
<code>HttpRequestMethodNotSupportedException</code>	Right path, wrong method	405
<code>HttpMediaTypeNotSupportedException</code>	Wrong <code>Content-Type</code>	415
<code>NoResourceFoundException</code>	No handler for the path	404
<code>DataIntegrityViolationException</code>	A database constraint rejected the write	409 is the useful choice

The last one is worth a handler of its own. Left alone it becomes a 500, which tells a client the server broke when in fact the request was wrong.

```
@ExceptionHandler(DataIntegrityViolationException.class)
public ProblemDetail handleConflict(DataIntegrityViolationException e) {
    return ProblemDetail.forStatusAndDetail(
        HttpStatus.CONFLICT, "That value is already taken");
}
```

The message deliberately says nothing about columns or constraint names. A database error text in a response tells an attacker the schema.

The simple case

A custom exception that only needs a status does not need a handler at all:

```
@ResponseStatus(HttpStatus.NOT_FOUND)
public class EmployeeNotFoundException extends RuntimeException { }
```

That is enough when the status is the whole answer. A handler is for when the body needs shaping.

Shaping the response

`ProblemDetail` carries more than a status and a message. A `type` URI points at documentation, and extra properties describe the specific failure:

```
@ExceptionHandler(InsufficientLeaveException.class)
public ProblemDetail handle(InsufficientLeaveException e) {
    ProblemDetail problem = ProblemDetail.forStatusAndDetail(
        HttpStatus.UNPROCESSABLE_ENTITY, e.getMessage());

    problem.setType(URI.create("https://errors.example.com/insufficient-leave"));
    problem.setTitle("Not enough leave");
    problem.setProperty("requested", e.getRequested());
    problem.setProperty("remaining", e.getRemaining());
    return problem;
}
```

A client can then branch on `type`, which is stable, rather than on the message, which is not.

Scoping and ordering

A `@RestControllerAdvice` applies to every controller by default, and can be narrowed:

```
@RestControllerAdvice(basePackages = "com.example.payroll.api")
@RestControllerAdvice(assignableTypes = EmployeeController.class)
```

When two handlers could both apply, **the most specific exception type wins**, not the first one declared. A handler for `Exception` therefore does not intercept the others, and is the right place for the last resort:

```
@ExceptionHandler(Exception.class)
public ProblemDetail handleUnexpected(Exception e) {
    String id = UUID.randomUUID().toString();
    log.error("unexpected failure {}", id, e);

    ProblemDetail problem = ProblemDetail.forStatusAndDetail(
        HttpStatus.INTERNAL_SERVER_ERROR, "Unexpected error");
    problem.setProperty("errorId", id);
    return problem;
}
```

The client gets an identifier and nothing else. The log gets everything.

What the advice cannot catch

`@RestControllerAdvice` runs inside Spring MVC. Anything failing before the dispatcher is reached never gets there:

Where it fails	Handled by the advice
A controller or service	Yes
A <code>@Validated</code> service method	Yes
A servlet filter, including the security chain	No
An <code>@Async</code> method, after the response was sent	No
A <code>@Scheduled</code> task	No

The security filter chain is the one that surprises people. A rejected token produces a 401 from `ExceptionTranslationFilter`, not from the advice, so matching the error shape across both needs an `AuthenticationEntryPoint`.

An `@Async` method has no response left to write to, so its failures belong in a log and a metric, with an `AsyncUncaughtExceptionHandler` to make sure they are not lost silently.

8. Testing

Two levels, and they answer different questions.

`@WebMvcTest` starts the web layer only, with the service mocked. It is fast and it tests the HTTP contract:

```
@WebMvcTest(EmployeeController.class)
class EmployeeControllerTest {

    @Autowired MockMvc mockMvc;
    @MockitoBean EmployeeService service;

    @Test
    void returnsNotFoundWhenMissing() throws Exception {
        when(service.findById(99L)).thenReturn(new EmployeeNotFoundException(99L));

        mockMvc.perform(get("/api/v1/employees/99"))
            .andExpect(status().isNotFound());
    }

    @Test
    void rejectsAnInvalidEmail() throws Exception {
        mockMvc.perform(post("/api/v1/employees")
            .contentType(MediaType.APPLICATION_JSON)
            .content("""
                { "name": "Ravi", "email": "nope", "salary": 75000 }
            """))
            .andExpect(status().isBadRequest());
    }
}
```

`@MockitoBean` replaced `@MockBean` in Spring Boot 3.4.

The second level starts everything against a real database in a container:

```
@SpringBootTest
@AutoConfigureMockMvc
@Testcontainers
class EmployeeApiIT {

    @Container
    static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>
("postgres:16");

    @DynamicPropertySource
    static void datasource(DynamicPropertyRegistry registry) {
        registry.add("spring.datasource.url", postgres::getJdbcUrl);
        registry.add("spring.datasource.username", postgres::getUsername);
        registry.add("spring.datasource.password", postgres::getPassword);
    }
}
```

Test	Starts	Answers
Plain unit test	Nothing	Does this method's logic hold
<code>@WebMvcTest</code>	The web layer	Right status, right JSON, validation applied
<code>@DataJpaTest</code>	JPA and a database	Do the queries and mappings work
<code>@SpringBootTest</code> with Testcontainers	Everything	Does the whole path work

Test against the real database, not H2. H2 accepts SQL that PostgreSQL rejects, so a passing build can still fail on deployment. A container costs a few seconds and removes that class of surprise.

9. Making it production ready

The last pieces are what separate a working endpoint from a service a team can operate.

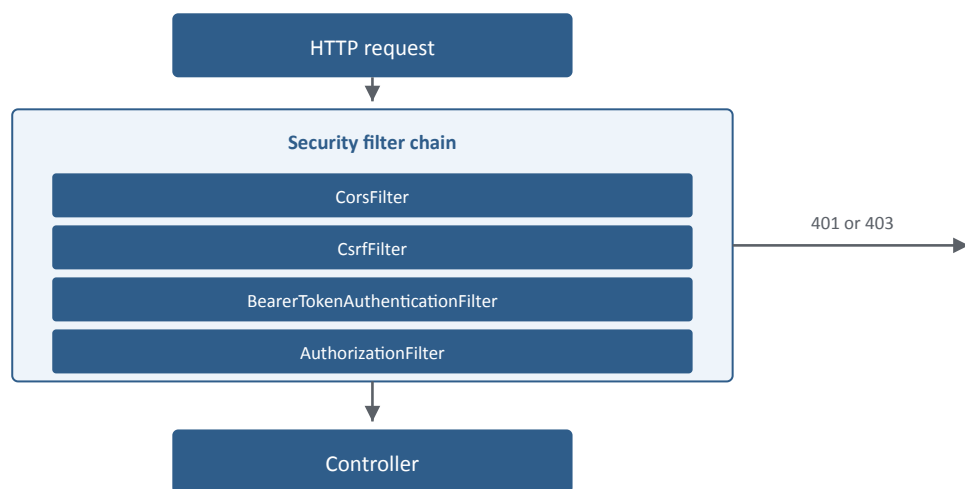
Piece	What to do
Schema	Flyway or Liquibase. Never <code>ddl-auto=update</code> in a shared environment
Documentation	springdoc-openapi generates OpenAPI and a UI from the controllers
Health	<code>/actuator/health</code> for the liveness and readiness probes
Configuration	One file per profile, and secrets from the environment
Logging	Structured JSON with a correlation id on every line
Pagination	Every list endpoint, from the first version

```
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
</dependency>
```

That one dependency reads the controllers and the DTOs and publishes the schema at `/v3/api-docs`, with a browsable page at `/swagger-ui.html`. Because it is generated, it cannot drift from the code.

10. Where Spring Security sits

Spring Security is a chain of servlet filters placed in front of the application. A request passes through the whole chain before any controller method runs.



Both authentication and authorization finish before the controller is reached. A rejected request never arrives.

By the time a controller method runs, the caller is known and allowed. That is why none of the controller code in this document mentions security.

`AuthorizationFilter` is last in the chain, so anything a filter rejects earlier never reaches the access rules.

11. Configuring it

Security is configured by publishing a `SecurityFilterChain` bean. `WebSecurityConfigurerAdapter` was removed in Spring Security 6, so any example that extends it is out of date.

```

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        return http
            .csrf(csrf -> csrf.disable())
            .sessionManagement(session ->

session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/actuator/health").permitAll()
                .requestMatchers(HttpMethod.GET, "/api/v1/employees/**")
                    .hasAuthority("SCOPE_employees.read")
                .requestMatchers("/api/v1/employees/**")
                    .hasAuthority("SCOPE_employees.write")
                .anyRequest().authenticated())
            .oauth2ResourceServer(oauth2 ->
oauth2.jwt(Customizer.withDefaults()))
            .build();
    }
}

```

Every line there is a decision:

Line	What it decides
<code>csrf.disable()</code>	Correct for a token API, wrong for anything using cookies
<code>STATELESS</code>	No session is created, and none is looked for
<code>permitAll()</code> on health	The probe has to work before anyone signs in
Two <code>requestMatchers</code>	Reading and writing need different scopes
<code>anyRequest().authenticated()</code>	Anything not listed is closed, not open

The order of the rules is the order they are checked, and the first match wins. Putting a broad rule above a narrow one makes the narrow one unreachable:

```

.requestMatchers("/api/**").permitAll() // matches everything
.requestMatchers("/api/v1/employees/**").authenticated() // never reached

```

Ending with `anyRequest().authenticated()` is what makes the configuration closed by default. Without it, an endpoint added later with no matching rule is open.

12. Running as a resource server

A resource server is an API that accepts tokens issued elsewhere. One property configures it:

```
spring.security.oauth2.resourceserver.jwt.issuer-  
uri=https://auth.example.com/realms/payroll
```

From that, Spring fetches the issuer's metadata, finds its public keys, and builds a `JwtDecoder`. Every request then has its token checked for signature, expiry and issuer before the controller sees it. No token handling code is written.

By default a `scope` claim becomes authorities with a `SCOPE_` prefix, which is where `hasAuthority("SCOPE_employees.read")` above comes from. An authorization server that puts roles in a different claim needs a converter:

```
@Bean  
JwtAuthenticationConverter jwtAuthenticationConverter() {  
    JwtGrantedAuthoritiesConverter authorities = new  
    JwtGrantedAuthoritiesConverter();  
    authorities.setAuthoritiesClaimName("roles");  
    authorities.setAuthorityPrefix("ROLE_");  
  
    JwtAuthenticationConverter converter = new JwtAuthenticationConverter();  
    converter.setJwtGrantedAuthoritiesConverter(authorities);  
    return converter;  
}
```

The token itself is available as a parameter when a claim is needed:

```
@GetMapping("/me")  
public Map<String, Object> me(@AuthenticationPrincipal Jwt jwt) {  
    return Map.of(  
        "subject", jwt.getSubject(),  
        "expiresAt", jwt.getExpiresAt());  
}
```

13. Users and passwords

When there is no external authorization server, Spring needs two beans: one that finds a user, and one that hashes passwords.

```

@Service
public class DatabaseUserDetailsService implements UserDetailsService {

    private final UserRepository repository;

    @Override
    public UserDetails loadUserByUsername(String email) {
        return repository.findByEmail(email)
            .map(user -> User.withUsername(user.getEmail())
                .password(user.getPasswordHash())
                .authorities(user.getRoles().toArray(new String[0]))
                .build())
            .orElseThrow(() -> new UsernameNotFoundException(email));
    }
}

```

```

@Bean
PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}

```

`PasswordEncoder` is one-way. Spring compares a submitted password by hashing it and comparing hashes, and there is no method that returns the original. A column called `password` rather than `passwordHash` is usually a sign something is wrong.

The authenticated user is held in a `ThreadLocal` :

```

Authentication authentication =
    SecurityContextHolder.getContext().getAuthentication();

```

That works inside a request and not in a thread started from one. An `@Async` method sees an empty context unless the strategy is set to propagate it. Taking the principal as a parameter avoids the problem entirely.

14. Method security

URL rules are coarse. A rule about a particular object belongs on the method that handles it.

```

@Configuration
@EnableMethodSecurity
public class MethodSecurityConfig { }

```

```
@PreAuthorize("hasAuthority('SCOPE_employees.write')")
public EmployeeResponse create(EmployeeRequest request) { ... }

@PreAuthorize("hasRole('ADMIN') or #email == authentication.name")
public EmployeeResponse findByEmail(String email) { ... }
```

The second rule cannot be written as a URL pattern, because it compares an argument against the caller.

Where	Good for
<code>authorizeHttpRequests</code> in the chain	Broad rules by path and method
<code>@PreAuthorize</code> on a service method	Rules that depend on the arguments or the data

`@PreAuthorize` is applied by a proxy, exactly like `@Transactional`. A call from another method in the same class skips it.

15. Testing it

`spring-security-test` adds the request builders and annotations.

```

@WebMvcTest(EmployeeController.class)
@Import(SecurityConfig.class)
class EmployeeSecurityTest {

    @Autowired MockMvc mockMvc;
    @MockitoBean EmployeeService service;

    @Test
    void rejectsAnUnauthenticatedRequest() throws Exception {
        mockMvc.perform(get("/api/v1/employees"))
            .andExpect(status().isUnauthorized());
    }

    @Test
    void allowsAReadScope() throws Exception {
        mockMvc.perform(get("/api/v1/employees")
            .with(jwt().authorities(
                new SimpleGrantedAuthority("SCOPE_employees.read"))))
            .andExpect(status().isOk());
    }

    @Test
    void refusesToWriteWithOnlyAReadScope() throws Exception {
        mockMvc.perform(post("/api/v1/employees")
            .with(jwt().authorities(
                new SimpleGrantedAuthority("SCOPE_employees.read")))
            .contentType(MediaType.APPLICATION_JSON)
            .content("""
                { "name": "Ravi", "email": "ravi@example.com", "salary":
75000 }
                """))
            .andExpect(status().isForbidden());
    }
}

```

Those three tests are the ones worth writing for every protected endpoint: without a token, with the wrong authority, and with the right one.

Note that `@Import(SecurityConfig.class)` is required. `@WebMvcTest` does not load the security configuration on its own, and a test that omits it passes while proving nothing.

Spring uses the two rejections differently. A missing or invalid token produces `401`, and a valid token without the required authority produces `403`. A test expecting the wrong one of those usually means the rule is in the wrong place.

The pieces together

Piece	Its job
<code>@RestController</code>	Maps HTTP to methods, and nothing else
Request and response records	The API shape, kept apart from the table
<code>@Valid</code>	Rejects a bad request before any logic runs
<code>@Service</code> with <code>@Transactional</code>	The rules, and the transaction boundary
<code>JpaRepository</code>	Data access
<code>@RestControllerAdvice</code> with <code>ProblemDetail</code>	One error shape for every endpoint
<code>@WebMvcTest</code> , Testcontainers	The contract, then the whole path
<code>SecurityFilterChain</code>	Which paths need which authority
<code>oauth2ResourceServer</code>	Token checking, from one property
<code>@PreAuthorize</code>	A rule that depends on the arguments

For example, a new endpoint for departments starts as a record for the request and one for the response, a repository interface, a service holding the rules and the transaction, and a controller that maps five methods onto them. The exception handler already exists, so a `DepartmentNotFoundException` becomes a 404 without touching the controller. The OpenAPI page updates itself. The tests are one `@WebMvcTest` for the contract and one container test for the path.

The most useful thing at this point is not to memorize the annotations. Take an endpoint that returns an entity directly, add the two records and the mapper, and see which fields were being exposed that nobody meant to publish.